



॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS
INSTITUTE OF TECHNOLOGY, NAGPUR
Practical File Submission

Course Name: JavaScript Lab

Course Code: T-3013 / 0705210508

Submitted by

Anaya Shende

24070521213

Batch: 2024-28

Semester - V

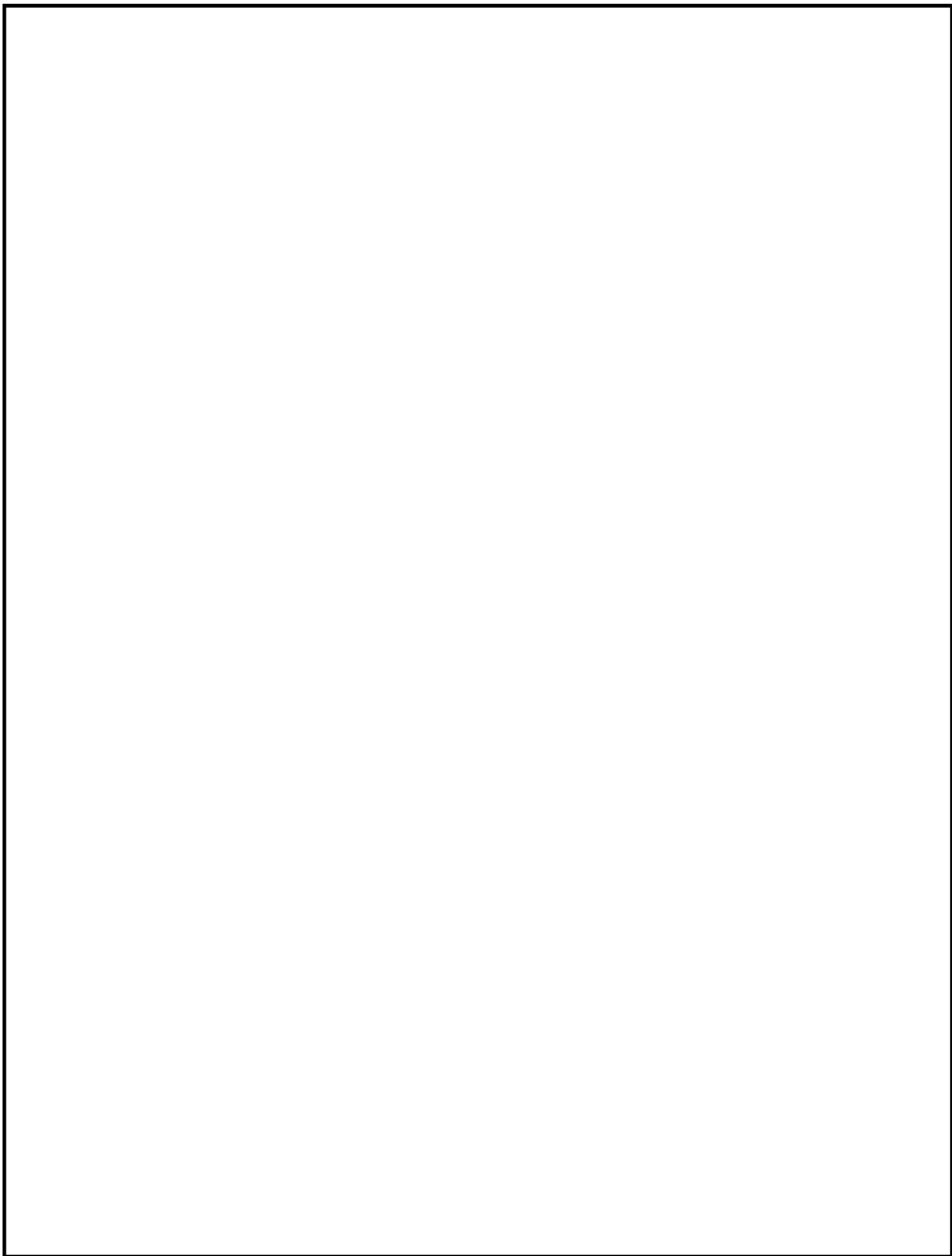
Section: A/B/C/D

Submitted to

Dr. Shreyas Rajendra Hole

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under Section 3 of the UGC Act, 1956) | Re-accredited by NAAC with 'A++' grade | Awarded Category - I by UGC



Certificate

Department of Computer Science and Engineering

This is to certify that this Practical Record contains the bona fide practical work
carried out by **Mr./Ms.** _____,
bearing **PRN.** _____, of **Batch** _____, **Semester** _____,
Section _____, during the **Academic Year** _____.

Date: _____

Name of Course Teacher: _____

Signature of Course Teacher: _____

Name of Student: _____ PRN: _____

Batch: _____ Semester: _____ Section: _____

Index

Practical No.	Aim	Performance Date	Submission Date	Remarks
<i>1</i>	Demonstrate inline, internal, and external JavaScript; use console methods; create a webpage showing user info and a welcome message.			
<i>2</i>	Use var, let, const, and demonstrate basic features like template literals and destructuring; build a billing calculator with user input.			
<i>3</i>	Implement control structures and form validation; create a grading system based on user-entered marks.			
<i>4</i>	Use function types, scope, and closures; apply try-catch; build a palindrome checker.			
<i>5</i>	Apply array methods and object handling; create a cart total calculator with discount logic.			
<i>6</i>	Use string functions and regex for email validation, data extraction, and text analysis.			
<i>7</i>	Perform DOM traversal and updates; develop a to-do list app with add, edit, and delete features.			
<i>8</i>	Access and validate form fields using events; design a gym admission form with live input checks.			
<i>9</i>	Implement localStorage and sessionStorage to save user preferences like theme settings.			
<i>10</i>	Load and display JSON data using fetch() and jQuery \$.getJSON(); render results dynamically in a table.			
<i>VA 11</i>	Use JavaScript timers (setTimeout(), setInterval()) and ES6 modules (import/export) to build a dynamic countdown or class scheduler with real-time updates and responsive layout.			
<i>VA 12</i>	Build an interactive real-time web app using JavaScript and jQuery with DOM, events, storage, and AJAX integration.			



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

EXPERIMENT NO: 1

Title : Demonstrate inline, internal, and external JavaScript; use console methods; create a webpage showing user info and a welcome message.

2. Software/Tools Required: Visual Studio Code, Google Chrome, HTML5, CSS3, and JavaScript.

3. Theory: JavaScript is a scripting language used to make web pages interactive and dynamic. It can be added to an HTML document in three different ways:

1. Inline JavaScript:

Inline JavaScript is written directly inside an HTML element using event attributes such as `onclick`. In this experiment, it was used to perform an action directly when the user interacted with a webpage element.

2. Internal JavaScript:

Internal JavaScript is written inside the `<script>` tag within the same HTML file. In this experiment, JavaScript functions were defined internally and called when specific user interactions occurred on the webpage.

3. External JavaScript:

External JavaScript is written in a separate `.js` file and connected to the HTML document using the `<script src="..."></script>` tag. In this experiment, external JavaScript was used to keep the scripting code separate from the HTML structure.

CODE :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>palindrome Checker</title>
  <style>
    body {
      font-family: monospace;
      background-color: pink;
      color: black;
      padding: 50px;
    }

    h1 {
      font-size: 20px;
      text-transform: uppercase;
    }
    h2 {
      font-size: 60px;
      font-style: italic;
      color: black;
    }

    .box {
      background-color: black;
      color: white;
      padding: 20px;
      width: 600px;
      border: 4px solid black;
    }

    input {
      width: 90%;
      padding: 8px;
      margin-bottom: 10px;
      background-color: white;
      color: black;
      border: 2px solid pink;
    }

    button {
```

```

        width: 97%;
        padding: 10px;
        background-color: pink;
        color: black;
        font-weight: bold;
        border: 2px solid white;
        cursor: pointer;
    }

    .result-box {
        margin-top: 15px;
        padding: 10px;
        border: 2px dashed pink;
        display: none;
    }
</style>
</head>
<body>

<div class="box">
    <h1>Checker</h1>
    <p>Is it a palindrome?</p>

    <input type="text" id="wordInput" placeholder="Type here...">
    <button id="submitBtn">CHECK</button>

    <div id="outputDisplay" class="result-box"></div>
    <p id="totalChecks">Checks: 0</p>
</div>
<h2> 24070521213</h2>
<script>
    const submitBtn = document.getElementById('submitBtn');
    const wordInput = document.getElementById('wordInput');
    const outputDisplay = document.getElementById('outputDisplay');
    const totalChecks = document.getElementById('totalChecks');

    function makeCounter() {
        let count = 0;
        return function() {
            count++;
            return count;
        };
    }
    const incrementChecks = makeCounter();

```

```
function basicClean(str) {
  return str.toLowerCase().replace(/[^a-z0-9]/g, "");
}

const flipString = function(str) {
  return str.split("").reverse().join("");
};

const checkMatch = (a, b) => a === b;

submitBtn.addEventListener('click', function() {
  outputDisplay.style.display = "none";

  try {
    const textValue = wordInput.value;

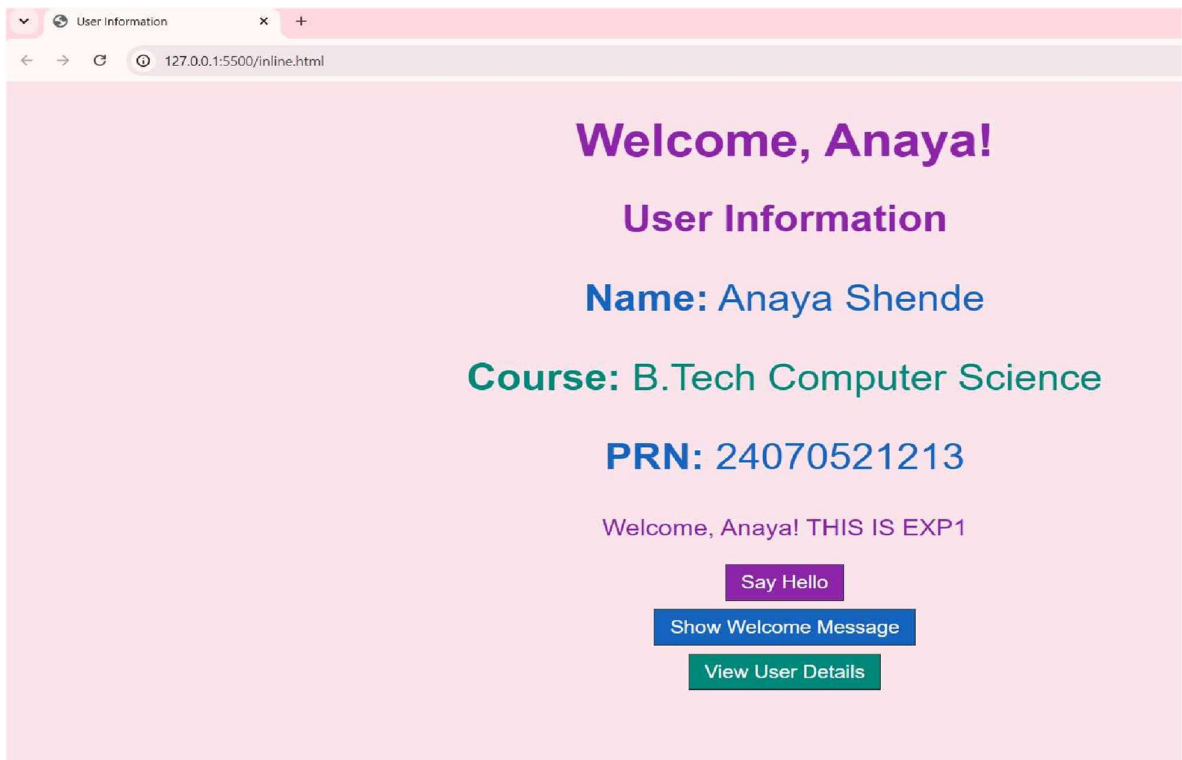
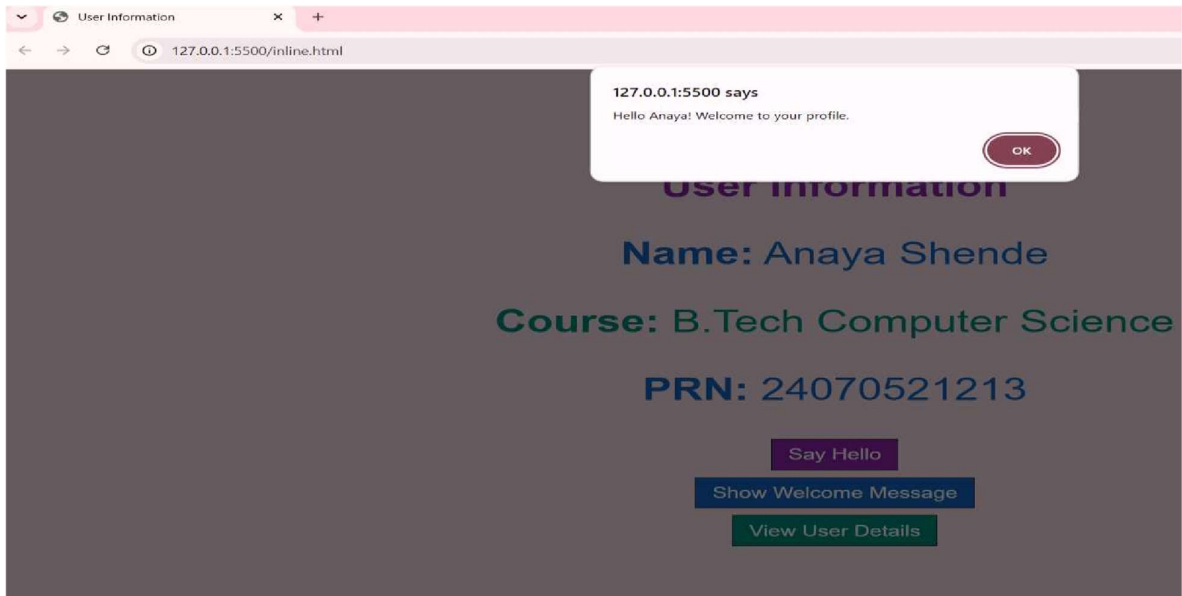
    if (textValue.trim() === "") {
      throw new Error("Type a word first!");
    }

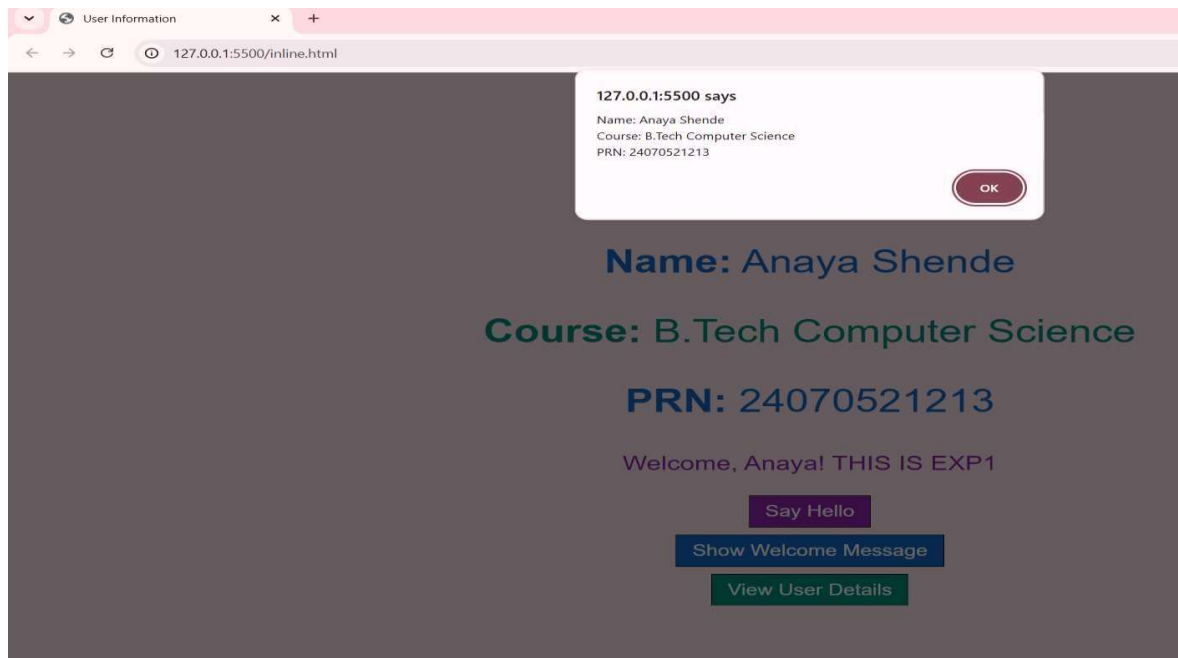
    const clean = basicClean(textValue);
    const reversed = flipString(clean);
    const isTrue = checkMatch(clean, reversed);

    const currentCount = incrementChecks();
    totalChecks.textContent = "Checks: " + currentCount;

    outputDisplay.style.display = "block";
    if (isTrue) {
      outputDisplay.textContent = "Yes! Matches backward.";
    } else {
      outputDisplay.textContent = "No! Does not match.";
    }
  } catch (err) {
    outputDisplay.style.display = "block";
    outputDisplay.textContent = "Error: " + err.message;
  }
});
</script>
</body>
</html>
```

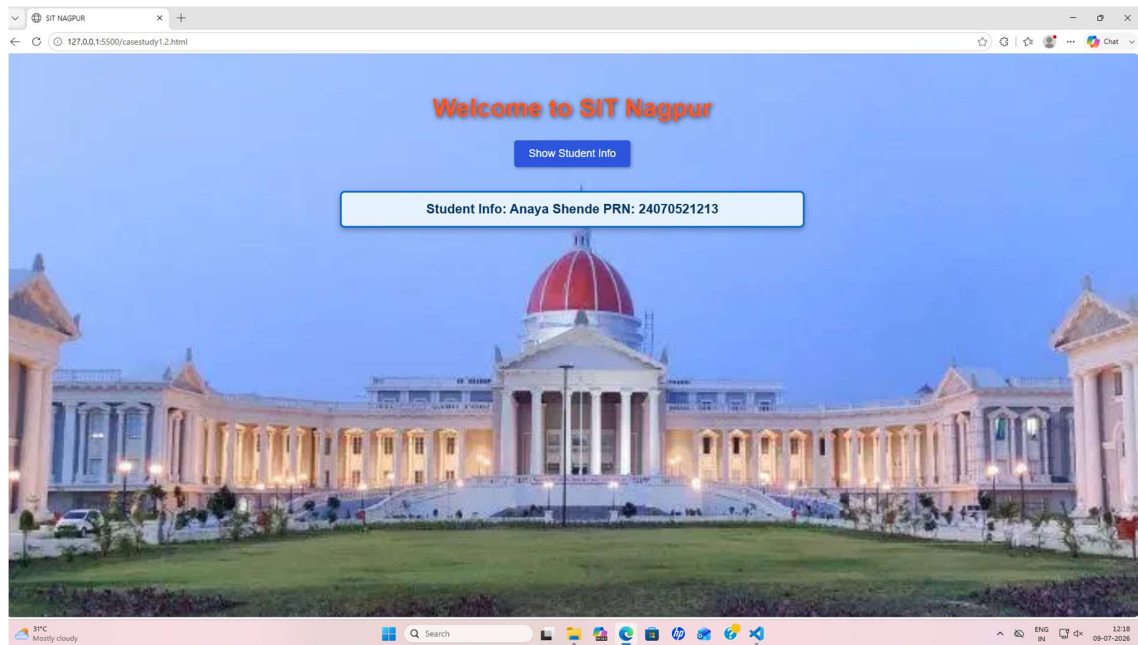
4. OUTPUT :





Conclusion : Thus, a webpage was successfully created using HTML, CSS, and JavaScript. The three methods of integrating JavaScript inline, internal, and external were successfully implemented and demonstrated through different interactive features on the webpage.

CASE STUDY – 1





SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

EXPERIMENT NO – 2

1. Title

Use var, let, const, and demonstrate basic features like template literals and destructuring; build a billing calculator with user input.

2. Software/Tools Required

Visual Studio Code (VS Code) , Google Chrome / Any Web Browser , HTML5, JavaScript (ES6)

3. Theory

JavaScript is a client-side scripting language used to create interactive web pages. In this experiment, a Billing Calculator is developed to calculate the total bill, GST, and final amount based on user input.

The following JavaScript features are demonstrated:

- **var:** Used for function-scoped variables and can be redeclared.
- **let:** Used for block-scoped variables whose values can be updated.
- **const:** Used for constant values that cannot be reassigned, such as the GST rate.
- **Template Literals:** Allow variables and expressions to be embedded inside strings using backticks (` ) and `${}` for easy formatting.
- **Destructuring:** Extracts values from an object into individual variables, making the code shorter and more readable.

The program accepts customer details, product name, quantity, and price, calculates the total amount, GST (18%), and final bill, then displays the receipt in a formatted manner.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Billing Calculator</title>
</head>
<body>

  <h2>Billing Receipt</h2>

  <pre id="receiptBox" style="background: #f0f0f0; padding: 15px; font-family: monospace;
font-size: 16px; width: 320px; border: 1px solid #ccc;"></pre>

<script>

  const GST_RATE = 0.18;

  var customerData = {
    customerName: prompt("Enter Customer Name:")
  };

  var productData = {
    productName: prompt("Enter Product Name:"),
    quantity: parseInt(prompt("Enter Quantity:")),
    pricePerItem: parseFloat(prompt("Enter Price Per Item (₹):"))
  };

  var { customerName } = customerData;
  var { productName, quantity, pricePerItem } = productData;

  let total = quantity * pricePerItem;
  let gstAmount = total * GST_RATE;
  let finalBill = total + gstAmount;

  var receiptText = `
```

=====

BILLING RECEIPT

=====

Customer Name : \${customerName}
Product Name : \${productName}
Quantity : \${quantity}
Price / Item : ₹\${pricePerItem.toFixed(2)}

Total : ₹\${total.toFixed(2)}
GST (18%) : ₹\${gstAmount.toFixed(2)}

FINAL BILL : ₹\${finalBill.toFixed(2)}

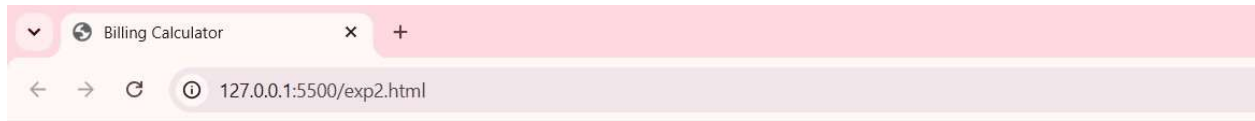
=====`;

```
// Immediately push the text into our HTML container on page load
document.getElementById("receiptBox").innerText = receiptText;
</script>
```

```
</body>
```

```
</html>
```

4.OUTPUT -



Billing Receipt

```
=====
                        BILLING RECEIPT
=====
Customer Name : Anaya Shende
Product Name  : laptop
Quantity      : 4
Price / Item  : ₹80000.00
-----
Total         : ₹320000.00
GST (18%)     : ₹57600.00
-----
FINAL BILL    : ₹377600.00
=====
```

ANAYA SHENDE

PRN: 24070521213

5. Result/Conclusion

The Billing Calculator was successfully implemented using JavaScript. The program accepted user input, calculated the total amount, GST (18%), and final bill accurately. The experiment successfully demonstrated the use of **var**, **let**, **const**, **template literals**, and **object destructuring** while generating a formatted billing receipt.

CASE STUDY 2

✓

SIT NAGPUR

×

Shopping Bill

×

+

← ↻ ⓘ 127.0.0.1:5500/casestudy2.html

Shopping Bill

Customer Name:

Anna

Phone Number:

7456765734

Product Name:

moblie

Quantity:

2

Rate in Rupee:

100000

Discount Amount (₹):

239

GST (%):

18

Packing Charges (₹):

45

Payment Mode:

Card ▼

Membership:

No ▼

Calculate Total Bill

INVOICE DETAILS

Invoice No: INV-6929
Customer: Anna
Phone: 7456765734
Product: moblie
Quantity: 2
Payment Mode: Card
Membership Card: No

GST Charged (18%): ₹35956.98
Final Total Bill: ₹235762.98



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

EXPERIMENT - 03

Title- Implementation of Control Structures and Form Validation for an Academic Grading System

Software / Tools Used

HTML5 , CSS3 , JavaScript , Visual Studio Code (VS Code) , Google Chrome / Microsoft Edge Web Browser

Theory - Control structures are fundamental programming constructs used to control the flow of execution in a program. Conditional statements such as if, else if, and else are used to make decisions based on specific conditions.

Form validation is a technique used to verify user input before processing it. It ensures that the entered data follows the required format and falls within acceptable limits. Validation improves data accuracy and prevents incorrect information from being processed.

In this experiment, an Academic Entry Portal is developed using HTML, CSS, and JavaScript. The application validates student details such as name, roll number, and marks entered for multiple subjects. Invalid inputs generate error messages, while valid inputs are processed to calculate: Total Marks , Percentage , Grade

The grading system uses conditional control structures to assign grades based on the calculated percentage. This demonstrates the practical application of decision-making statements and input validation in web development.

CODE :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Academic report</title>
  <style>
    body {
      font-family: monospace;
      background-color: #ffffff;
      color: #111111;
      max-width: 400px;
      margin: 40px auto;
      padding: 0 20px;
      line-height: 1.5;
    }
    h2 {
      border-bottom: 1px solid #111111;
      padding-bottom: 5px;
      text-transform: uppercase;
      font-size: 1.1rem;
      margin-bottom: 20px;
      margin-top: 30px;
    }
    .form-group {
      margin-bottom: 12px;
    }
    label {
      display: block;
      font-weight: bold;
      margin-bottom: 4px;
      font-size: 0.85rem;
    }
    input {
      width: 100%;
      padding: 8px;
      border: 1px solid #666666;
      box-sizing: border-box;
      font-family: monospace;
    }
    input:focus {
      outline: none;
```

```

        border: 1px solid #111111;
    }
    input:not(:placeholder-shown):invalid {
        border-style: dashed;
        border-color: #ff0000;
    }
    button {
        width: 100%;
        background-color: #111111;
        color: #ffffff;
        border: none;
        padding: 10px;
        font-family: monospace;
        font-weight: bold;
        cursor: pointer;
        text-transform: uppercase;
        letter-spacing: 1px;
        margin-top: 15px;
    }
    button:hover {
        background-color: #444444;
    }
    .output-box {
        margin-top: 25px;
        padding: 15px;
        border: 1px solid #111111;
        display: none;
        white-space: pre-line; /* Preserves layout spaces and line breaks */
    }
    .error-message {
        color: #ff0000;
        font-size: 0.8rem;
        margin-top: 4px;
        display: none;
    }
}
</style>
</head>
<body>
    <h1>24070521213</h1>
    <h2>Academic Entry Portal</h2>
    <form id="gradeForm" novalidate>
        <div class="form-group">
            <label for="studentName">Student Name</label>
            <!-- Pattern ensures it contains only letters and spaces, matching real-world names -->

```

```
    <input type="text" id="studentName" placeholder="e.g. Alex Smith" required
minlength="2" maxlength="50" pattern="^[a-zA-Z\s]+$">
    <div class="error-message" id="nameError">Please enter a valid name using only
letters (min 2 characters).</div>
</div>
```

```
<div class="form-group">
    <label for="rollNo">Roll Number</label>
    <!-- Restricts input to numeric roll numbers only -->
    <input type="text" id="rollNo" placeholder="e.g. 101" required pattern="^[0-9]+$">
    <div class="error-message" id="rollError">Please enter a valid numeric roll
number.</div>
</div>
```

```
<div class="form-group">
    <label for="sub1">Subject 1 Marks (0 - 100)</label>
    <input type="number" id="sub1" placeholder="Score 1" required min="0"
max="100" step="0.1">
    <div class="error-message" id="sub1Error">Marks must be between 0 and
100.</div>
</div>
```

```
<div class="form-group">
    <label for="sub2">Subject 2 Marks (0 - 100)</label>
    <input type="number" id="sub2" placeholder="Score 2" required min="0"
max="100" step="0.1">
    <div class="error-message" id="sub2Error">Marks must be between 0 and
100.</div>
</div>
```

```
<div class="form-group">
    <label for="sub3">Subject 3 Marks (0 - 100)</label>
    <input type="number" id="sub3" placeholder="Score 3" required min="0"
max="100" step="0.1">
    <div class="error-message" id="sub3Error">Marks must be between 0 and
100.</div>
</div>
```

```
<div class="form-group">
    <label for="sub4">Subject 4 Marks (0 - 100)</label>
    <input type="number" id="sub4" placeholder="Score 4" required min="0"
max="100" step="0.1">
    <div class="error-message" id="sub4Error">Marks must be between 0 and
100.</div>
```

```
</div>
```

```
    <button type="submit">Process Report</button>
</form>
```

```
<h2>System Output Receipt</h2>
<div id="resultDisplay" class="output-box"></div>
```

```
<script>
```

```
    var reportForm = document.getElementById('gradeForm');
    var viewOutput = document.getElementById('resultDisplay');
```

```
    reportForm.addEventListener('submit', function(event) {
        event.preventDefault();
```

```
        let hasErrors = false;
        const errorDivs = document.querySelectorAll('.error-message');
        errorDivs.forEach(div => div.style.display = 'none');
```

```
        const nameField = document.getElementById('studentName');
        const nameValue = nameField.value.trim();
        const namePattern = /^[a-zA-Z\s]+$/;
        if (nameValue.length < 2 || nameValue.length > 50 || !namePattern.test(nameValue)) {
            document.getElementById('nameError').style.display = 'block';
            hasErrors = true;
        }
```

```
        const rollField = document.getElementById('rollNo');
        const rollValue = rollField.value.trim();
        const rollPattern = /^[0-9]+$/;
        if (rollValue === "" || !rollPattern.test(rollValue)) {
            document.getElementById('rollError').style.display = 'block';
            hasErrors = true;
        }
```

```
        const subjects = ['sub1', 'sub2', 'sub3', 'sub4'];
        const scores = {};
```

```
        subjects.forEach(subId => {
            const inputElement = document.getElementById(subId);
```

```
const scoreValue = parseFloat(inputElement.value);

if (isNaN(scoreValue) || scoreValue < 0 || scoreValue > 100) {
    document.getElementById(`${subId}Error`).style.display = 'block';
    hasErrors = true;
} else {
    scores[subId] = scoreValue;
}
});

if (hasErrors) {
    viewOutput.style.display = 'none';
    return;
}

// Calculations
const totalMarks = scores.sub1 + scores.sub2 + scores.sub3 + scores.sub4;
const percentage = totalMarks / 4;

let assignedGrade = "";
if (percentage >= 90) {
    assignedGrade = 'A';
} else if (percentage >= 80) {
    assignedGrade = 'B';
} else if (percentage >= 70) {
    assignedGrade = 'C';
} else if (percentage >= 60) {
    assignedGrade = 'D';
} else {
    assignedGrade = 'F';
}

// Generate Output Receipt
viewOutput.innerHTML = `
```

OFFICIAL ACADEMIC REPORT

STUDENT: \${nameValue.toUpperCase()}
ROLL NO: \${rollValue}

Subject 1: \${scores.sub1.toFixed(1)} / 100
Subject 2: \${scores.sub2.toFixed(1)} / 100
Subject 3: \${scores.sub3.toFixed(1)} / 100

Subject 4: $\frac{\text{\${scores.sub4.toFixed(1)}}}{100}$

TOTAL MARKS: $\frac{\text{\${totalMarks.toFixed(1)}}}{400}$

PERCENTAGE: $\text{\${percentage.toFixed(1)}\%}$

FINAL GRADE: $\text{\${assignedGrade}}$

=====

```
`;  
    viewOutput.style.display = 'block';  
});  
</script>  
</body>  
</html>
```

Output:

24070521213

ACADEMIC ENTRY PORTAL

Student Name
6751464gt
Please enter a valid name using only letters (min 2 characters).

Roll Number
720782777777
Please enter a valid numeric roll number.

Subject 1 Marks (0 - 100)
610000
Marks must be between 0 and 100.

Subject 2 Marks (0 - 100)
04443
Marks must be between 0 and 100.

Subject 3 Marks (0 - 100)
93

Subject 4 Marks (0 - 100)
90

PROCESS REPORT

SYSTEM OUTPUT RECEIPT

24070521213

ACADEMIC ENTRY PORTAL

Student Name
Araya Shende

Roll Number
24070521213

Subject 1 Marks (0 - 100)
67

Subject 2 Marks (0 - 100)
82

Subject 3 Marks (0 - 100)
93

Subject 4 Marks (0 - 100)
99

PROCESS REPORT

SYSTEM OUTPUT RECEIPT

```
*****
OFFICIAL ACADEMIC REPORT
*****
STUDENT: 8783873878
ROLL NO: 24070521213

SUBJECT 1: 67 / 100
SUBJECT 2: 82 / 100
SUBJECT 3: 93 / 100
SUBJECT 4: 99 / 100
*****
TOTAL MARKS: 341 / 400
PERCENTAGE: 85.25
FINAL GRADE: B
*****
```

Conclusion - The experiment was successfully implemented using Form validation ensured that only valid student details and marks were accepted, while control structures were used to calculate total marks, percentage, and assign grades. The system effectively demonstrates the use of conditional statements and input validation techniques for developing reliable and user-friendly web applications.

CASE STUDY 3

Scholarship Finder Project

127.0.0.1:5500/case%20study%203.html

Scholarship Checker

Student Name:
5rhghghv

Enter Subject Marks (0 - 100):

Subject 1:
Must be between 0 and 100

Subject 2:
934
Must be between 0 and 100

Subject 3:
89

Subject 4:
99

Check Eligibility

Scholarship Finder Project

127.0.0.1:5500/case%20study%203.html

Scholarship Checker

Student Name:
jane lara

Enter Subject Marks (0 - 100):

Subject 1:
97

Subject 2:
98

Subject 3:
89

Subject 4:
99

Check Eligibility

Result for jane lara

Average Score: 95.75

Eligibility Status: **Platinum Scholarship**

Form submissions evaluated: 1



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

EXPERIMENT 4

Aim :

To use function types, scope, and closures, apply try-catch exception handling, and build a palindrome checker.

Tools Used :

Visual Studio Code (VS Code), HTML5, CSS3, JavaScript (ES6), Google Chrome, Live Server.

Theory :

A palindrome is a word, number, or phrase that reads the same forward and backward. This experiment demonstrates the use of JavaScript function types, variable scope, closures, and exception handling. Function declarations, function expressions, and arrow functions are used to organize and reuse code efficiently. Variable scope controls the accessibility of variables, including global scope, local scope, and block scope. Closures allow an inner function to retain access to variables of its outer function even after the outer function has finished execution, making it useful for maintaining the count of palindrome checks. The `try...catch` statement is used to detect and handle runtime errors without terminating the program. The palindrome checking process involves accepting user input, converting it to lowercase, reversing the string, comparing the original string with the reversed string, displaying whether it is a palindrome, and updating the total number of checks.

CODE :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>palindrome Checker</title>
  <style>
    body {
      font-family: monospace;
      background-color: pink;
      color: black;
      padding: 50px;
    }

    h1 {
      font-size: 20px;
      text-transform: uppercase;
    }
    h2 {
      font-size: 60px;
      font-style: italic;
      color: black;
    }

    .box {
      background-color: black;
      color: white;
      padding: 20px;
      width: 600px;
      border: 4px solid black;
    }

    input {
      width: 90%;
      padding: 8px;
      margin-bottom: 10px;
      background-color: white;
      color: black;
      border: 2px solid pink;
```

```

    }

    button {
      width: 97%;
      padding: 10px;
      background-color: pink;
      color: black;
      font-weight: bold;
      border: 2px solid white;
      cursor: pointer;
    }

    .result-box {
      margin-top: 15px;
      padding: 10px;
      border: 2px dashed pink;
      display: none;
    }
  </style>
</head>
<body>

  <div class="box">
    <h1>Checker</h1>
    <p>Is it a palindrome?</p>

    <input type="text" id="wordInput" placeholder="Type here...">
    <button id="submitBtn">CHECK</button>

    <div id="outputDisplay" class="result-box"></div>
    <p id="totalChecks">Checks: 0</p>
  </div>
<h2> 24070521213</h2>
<script>
  const submitBtn = document.getElementById('submitBtn');
  const wordInput = document.getElementById('wordInput');
  const outputDisplay = document.getElementById('outputDisplay');
  const totalChecks = document.getElementById('totalChecks');

  function makeCounter() {
    let count = 0;

```

```
    return function() {
        count++;
        return count;
    };
}
const incrementChecks = makeCounter();

function basicClean(str) {
    return str.toLowerCase().replace(/^[^a-z0-9]/g, "");
}

const flipString = function(str) {
    return str.split("").reverse().join("");
};

const checkMatch = (a, b) => a === b;

submitBtn.addEventListener('click', function() {
    outputDisplay.style.display = "none";

    try {
        const textValue = wordInput.value;

        if (textValue.trim() === "") {
            throw new Error("Type a word first!");
        }

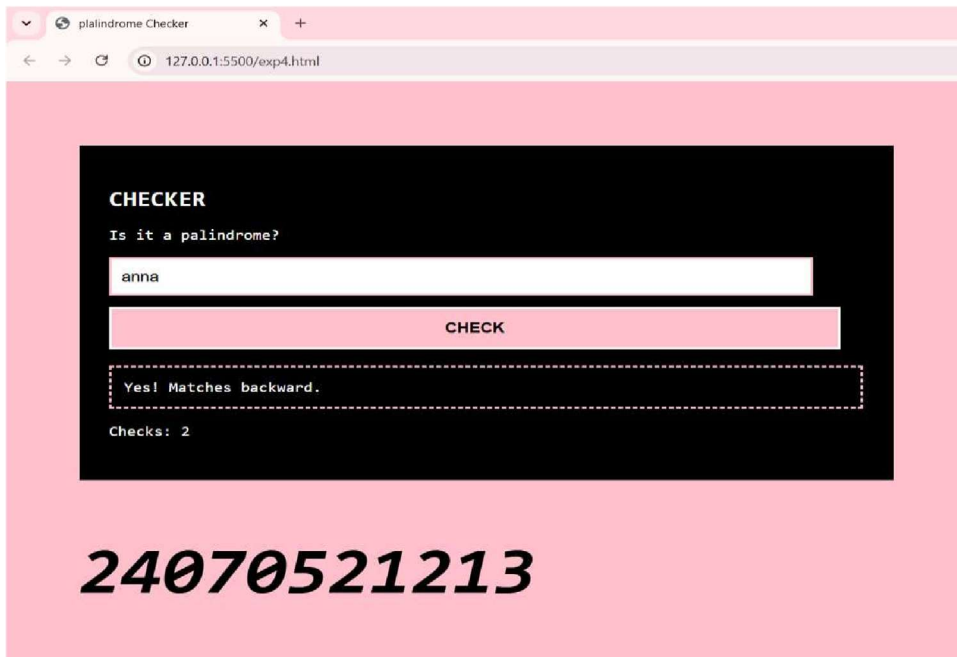
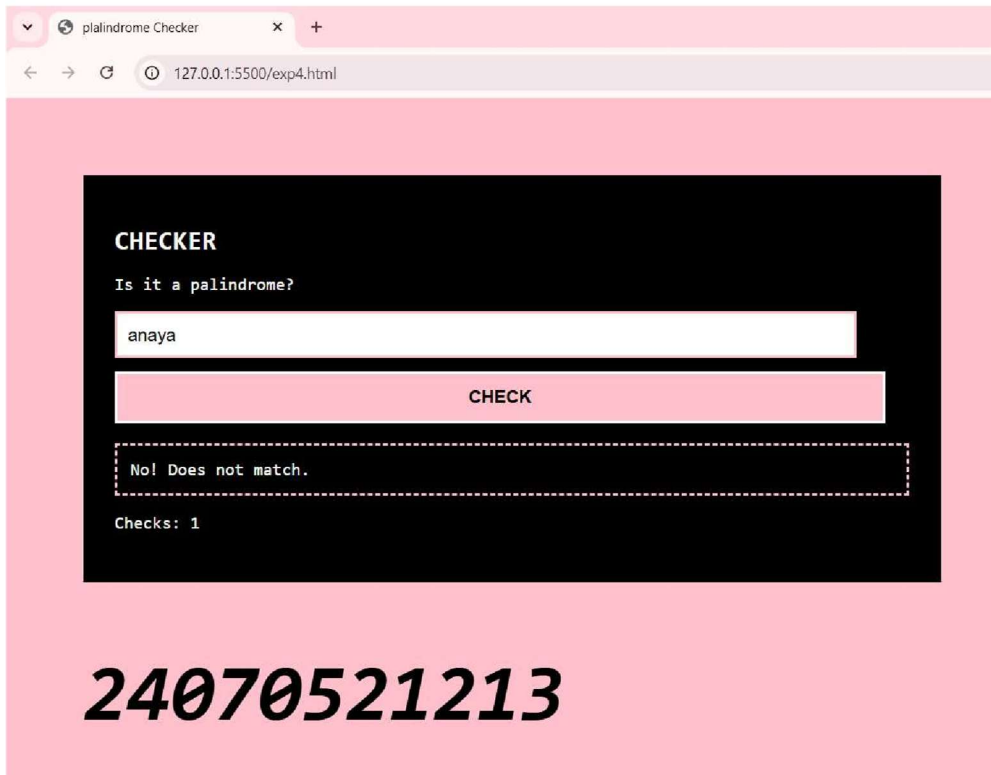
        const clean = basicClean(textValue);
        const reversed = flipString(clean);
        const isTrue = checkMatch(clean, reversed);

        const currentCount = incrementChecks();
        totalChecks.textContent = "Checks: " + currentCount;

        outputDisplay.style.display = "block";
        if (isTrue) {
            outputDisplay.textContent = "Yes! Matches backward.";
        } else {
            outputDisplay.textContent = "No! Does not match.";
        }
    }
});
```

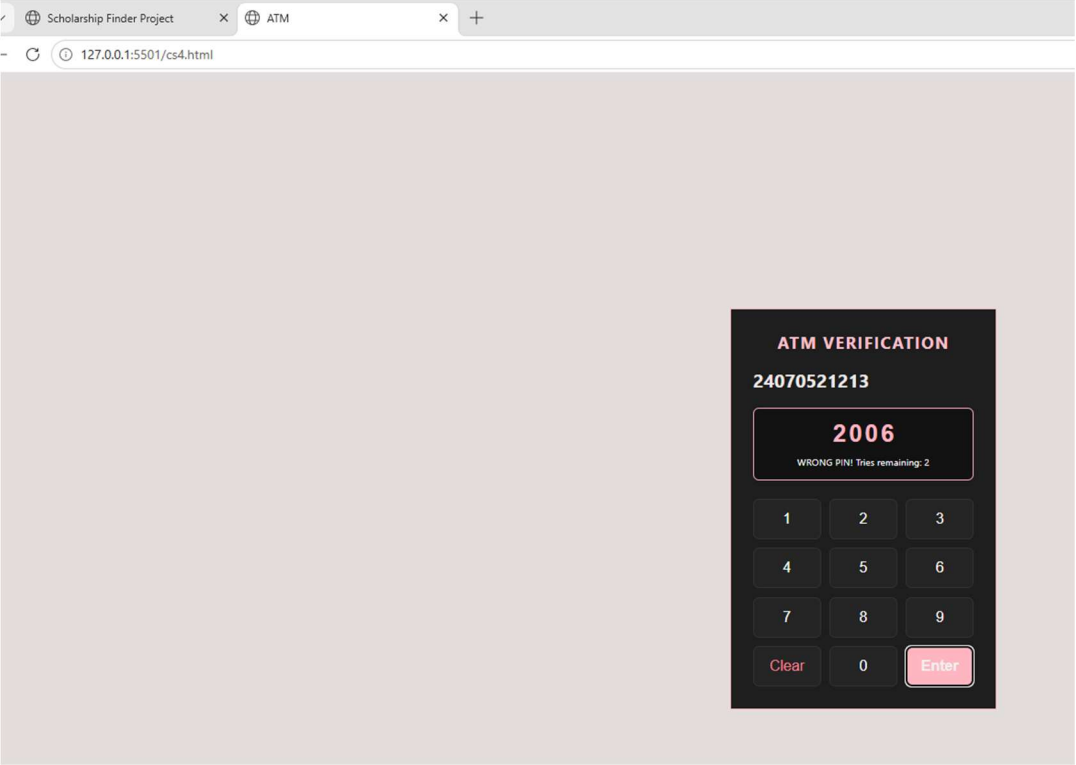
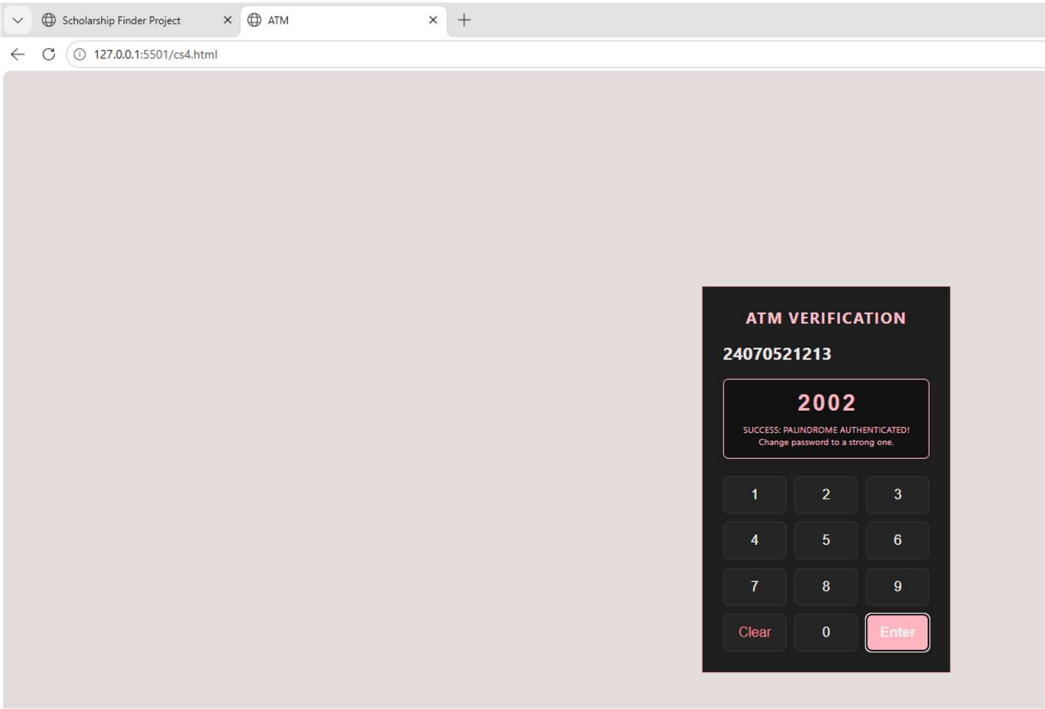
```
    } catch (err) {  
      outputDisplay.style.display = "block";  
      outputDisplay.textContent = "Error: " + err.message;  
    }  
  });  
</script>  
</body>  
</html>
```

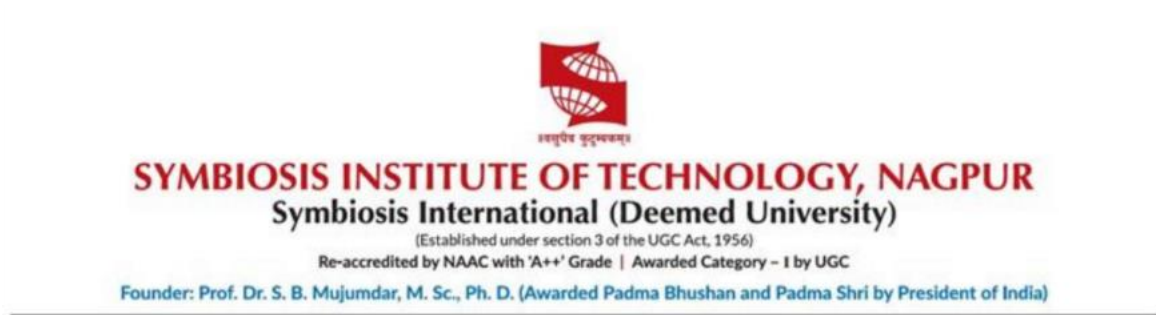
OUTPUT:



Conclusion : The palindrome checker was successfully developed using HTML, CSS, and JavaScript. It correctly identifies palindrome strings while demonstrating the use of function types, variable scope, closures, and try-catch exception handling.

CASE STUDY 4





EXPERIMENT - 5

Aim : Apply array methods and object handling; create a cart total calculator with discount logic.

Tools / Software Required: HTML, CSS, JavaScript, Visual Studio Code, Web Browser.

THEORY:

Array: An array is a collection of multiple values stored in a single variable. In this experiment, an array is used to store the products in the shopping cart.

Object: An object stores data in the form of **key-value pairs**. Each product is represented as an object containing its name, price, and quantity.

Array Methods: Array methods are built-in JavaScript functions used to perform operations on arrays. In this experiment, `map()` is used to display each product, while `reduce()` is used to calculate the total cost of all products.

Object Handling: Object handling involves creating, accessing, and modifying the properties of objects. The product objects are accessed to obtain the product name, price, and quantity.

Discount

The discount is calculated using the entered percentage.

Logic:

Discount Amount = (Subtotal × Discount %) / 100

Final Total = Subtotal – Discount Amount

OUTPUT :

SHOPPING CART

127.0.0.1:5500/exp5.html

Product Details

Item 1

TV

80000

2

+ Add Another Product

Discount

20

Generate Receipt

20% discount applied. Receipt generated!

SHOPPING RECEIPT

Customer: Anaya

Date: 8/10/2026

Item	Product	Qty	Price	Total
1	TV	2	₹60000.00	₹120000.00

Subtotal: ₹120000.00

Discount: ₹24000.00

Final Total: ₹96000.00

Thank you for shopping!

Print Receipt

SHOPPING CART

127.0.0.1:5500/exp5.html

...html

SHOPPING CART

24070521213

Customer Details

Customer Name:

Product Details

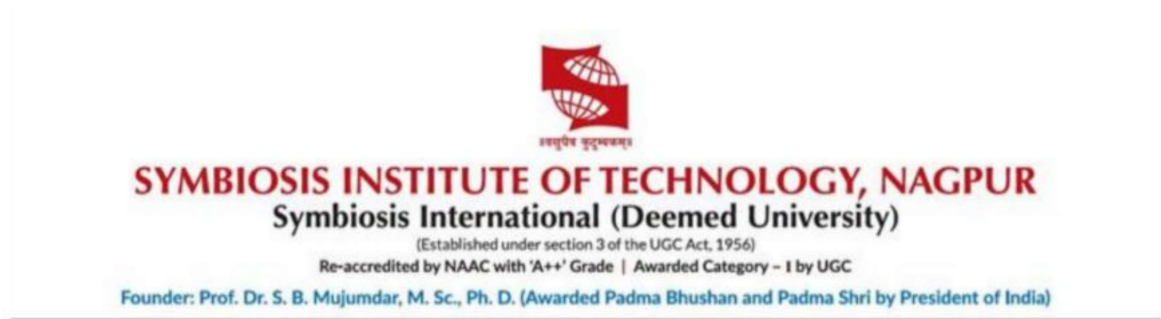
Item 1

+ Add Another Product

Discount

20% discount applied. Receipt generated!

CONCLUSION : Thus, the Cart Total Calculator was successfully developed using HTML, CSS, and JavaScript. The experiment demonstrated the use of arrays, objects, array methods and discount logic to calculate and generate a printable shopping receipt.



EXPERIMENT-6

AIM- Use of JavaScript string functions and regular expressions for email and phone validation, data extraction, and text analysis.

TOOLS USED - HTML5 , CSS3 , JavaScript , Regular Expressions (Regex) , VS Code ,Live Server , Google Chrome.

THEORY - The Student Portfolio Setup application demonstrates the use of JavaScript string manipulation and regular expressions to validate user input and analyze textual data on the client side.

String functions are used to process and modify text entered by the user. Functions such as `trim()`, `toUpperCase()`, `toLowerCase()`, `replace()`, `split()`, and `match()` help clean, format, and analyze user input. The application also calculates the number of characters, words, and sentences in the summary section, providing real-time text analytics.

Regular Expressions (Regex) are used to enforce input validation. The phone number field accepts only numeric values with exactly ten digits, while the email field verifies that the entered address follows a valid email format. Invalid inputs display error messages immediately without submitting the form.

The application performs local processing, meaning all validation, formatting, and analysis occur within the browser without requiring a server. After successful validation, the entered information is extracted, formatted, and displayed as a portfolio preview, demonstrating practical use of string processing, pattern matching, and data extraction techniques.

OUTPUT -

Student Portfolio Setup

127.0.0.1:5500/exp6.html

Ethics in Engineerin... Home | Infosys Spri...

Student Portfolio Setup

Enter your profile details. All string processing and structural validation happen locally.

FULL NAME

anaya

PHONE NUMBER (10 DIGITS)

vgfe45dcrf

Must contain only numeric values and exactly 10 digits.

EMAIL ADDRESS

anaya.shende@gamilin

Please enter a valid email format (e.g., user@domain.com).

SHORT SUMMARY ABOUT YOU

patent. Her work, titled "System and Method for Processor-Based Optimization of Gallium Nitride High Electron Mobility Transistor Structures" (Patent No: 202521127292), was officially published on January 2, 2026, marking a significant milestone in semiconductor-related research at her institution.

UPPERCASE

lowercase

Fix Double Spaces

Characters: 672

Words: 92

Sentences: 4

Submit Portfolio

Student Portfolio Setup

127.0.0.1:5500/exp6.html

Ethics in Engineerin... Home | Infosys Spri...

UPPERCASE

lowercase

Fix Double Spaces

Characters: 672

Words: 92

Sentences: 4

Submit Portfolio

Information Submitted Successfully

Your portfolio record has been verified, parsed, and logged.

Name: ANAYA SHENDE

Phone: 5654545764

Email: anaya.shende.batch2024@sitnagpur.siu.edu.in

Summary Analytics: 92 words | 672 characters | 4 sentences

Bio Paragraph:

Anaya Shende is an ambitious engineering student currently pursuing a Bachelor of Technology (B.Tech) in Computer Science and Engineering at the Symbiosis Institute of Technology (SIT) in Nagpur, as part of the 2024–2028 academic batch. Demonstrating an early aptitude for advanced research and technological innovation, she is already a co-inventor of a published utility patent. Her work, titled "System and Method for Processor-Based Optimization of Gallium Nitride High Electron Mobility Transistor Structures" (Patent No: 202521127292), was officially published on January 2, 2026, marking a significant milestone in semiconductor-related research at her institution.

Student Portfolio Setup

Enter your profile details. All string processing and structural validation happen locally.

FULL NAME

ANAYA SHENDE

PHONE NUMBER (10 DIGITS)

5654545764

EMAIL ADDRESS

anaya.shende.batch2024@sitnagpur.siu.edu.in

SHORT SUMMARY ABOUT YOU

patent. Her work, titled "System and Method for Processor-Based Optimization of Gallium Nitride High Electron Mobility Transistor Structures" (Patent No: 202521127292), was officially published on January 2, 2026, marking a significant milestone in semiconductor-related research at her institution.

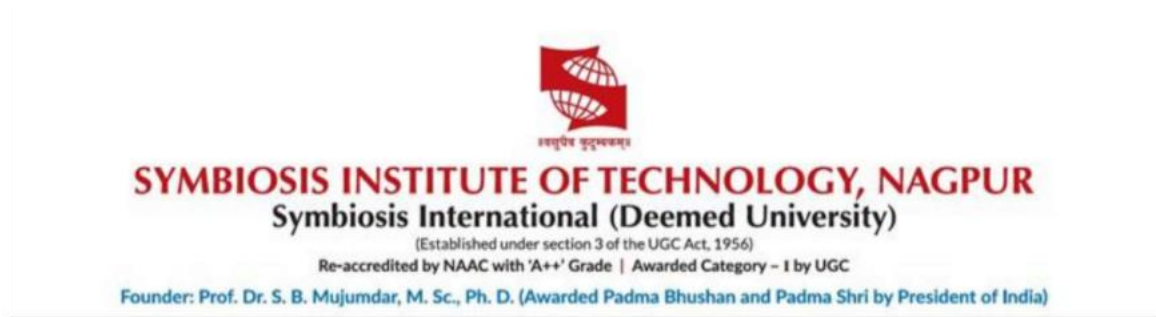
UPPERCASE lowercase Fix Double Spaces

Characters: 672 Words: 92 Sentences: 4

Submit Portfolio

Conclusion -

The Student Portfolio Setup application successfully demonstrates the implementation of JavaScript string functions and regular expressions for client-side input validation, text formatting, data extraction, and summary analysis. It effectively validates email addresses and phone numbers, processes user-entered text, generates useful analytics, and provides immediate feedback, making it a practical example of string processing and regex-based validation in web development.



Experiment 7

Aim - To perform DOM traversal and manipulation, and to develop a simple To-Do List application with **Add, Edit, and Delete** functionalities using HTML, CSS, and JavaScript.

Software Required - VS Code, Web Browser (Google Chrome), HTML, CSS, JavaScript

Theory -

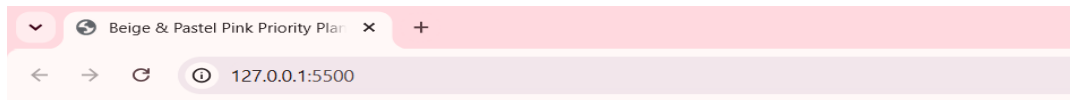
The **Document Object Model (DOM)** represents an HTML document as a tree of objects. JavaScript can access and modify these objects using DOM methods such as `getElementById()`, `querySelector()`, `createElement()`, and `appendChild()`.

DOM manipulation allows web pages to be updated dynamically without reloading the page. In a To-Do List application, JavaScript is used to create new task elements, modify existing tasks, and remove tasks from the DOM.

The main operations performed are:

- **Add:** Creates and displays a new task.
- **Edit:** Modifies the text or properties of an existing task.
- **Delete:** Removes a task element from the DOM.
- **DOM Traversal:** Accesses and navigates through HTML elements to perform these updates.

OUTPUT -

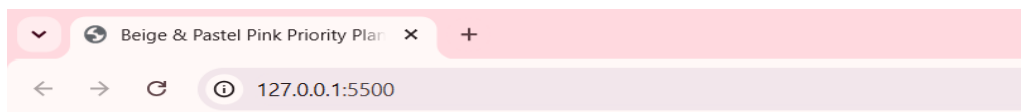


Task Planner

24070521213

⚡ UP NEXT

No pending tasks! Add one below.

Task Planner

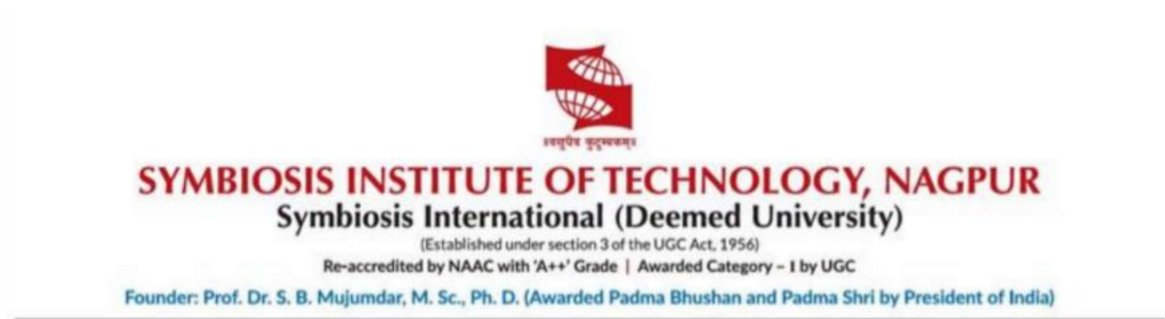
24070521213

⚡ UP NEXT

js exp 8 [Priority: High]

- js exp 7
Low
- js exp 8
High

Conclusion - The To-Do List application was successfully developed using HTML, CSS, and JavaScript. DOM traversal and manipulation were performed to dynamically **add, edit, and delete tasks** from the webpage.



EXPERIMENT – 9

Aim - Implement localStorage and sessionStorage to save user preferences such as theme settings.

Tools and Software Required - Visual Studio Code , Google Chrome / Web Browser , HTML5 , CSS3 , JavaScript , Live Server Extension in VS Code

Theory

Web Storage is a browser-based mechanism that allows websites to store data on the user's device. JavaScript provides two main types of Web Storage: **localStorage** and **sessionStorage**.

localStorage stores data persistently in the browser. The data remains available even after the browser or webpage is closed and reopened, until it is manually removed or cleared.

For example:

```
localStorage.setItem("theme", "pink"); let theme = localStorage.getItem("theme");
```

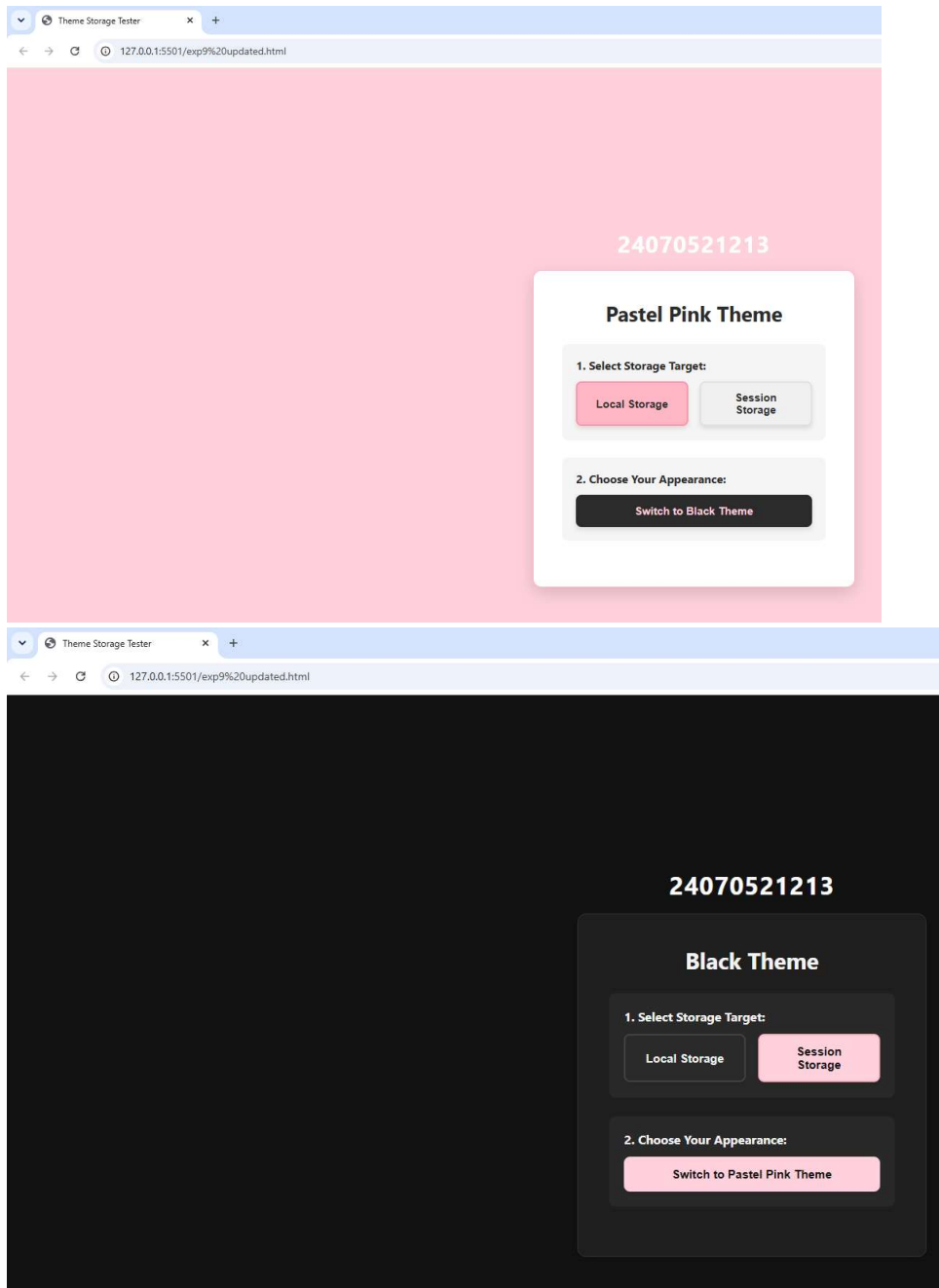
sessionStorage also stores data in the browser, but the data is associated with the current browser tab/session. It is generally cleared when the tab is closed.

```
sessionStorage.setItem("theme", "black"); let theme = sessionStorage.getItem("theme");
```

Both storage mechanisms store data as **key-value pairs**. The `setItem()` method is used to store data, `getItem()` retrieves stored data, and `removeItem()` removes a particular item.

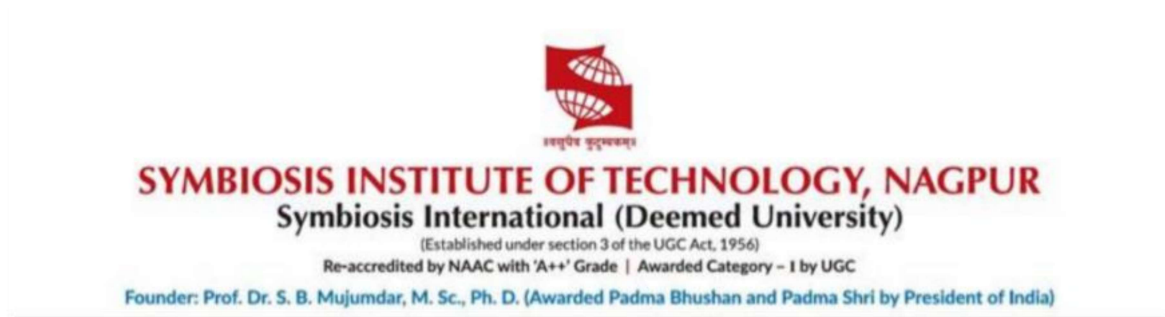
In this experiment, the user can select either Local Storage or Session Storage and choose a theme such as **Black Theme** or **Pastel Pink Theme**.

OUTPUT -



Conclusion

Thus, `localStorage` and `sessionStorage` were successfully implemented to store and retrieve user theme preferences. The experiment demonstrated the difference between persistent local storage and temporary session storage.



EXPERIMENT – 10

Aim - To load and display JSON data using `fetch()` and `jQuery.getJSON()`, and render the results dynamically in a table.

Tools and Software Required - Visual Studio Code , Google Chrome / Web Browser
HTML5 , CSS3 , JavaScript , jQuery Library , JSON file , Live Server Extension in VS Code .

Theory - JSON (JavaScript Object Notation) is a lightweight data format commonly used to store and exchange structured information. JSON represents data using objects and arrays containing key-value pairs.

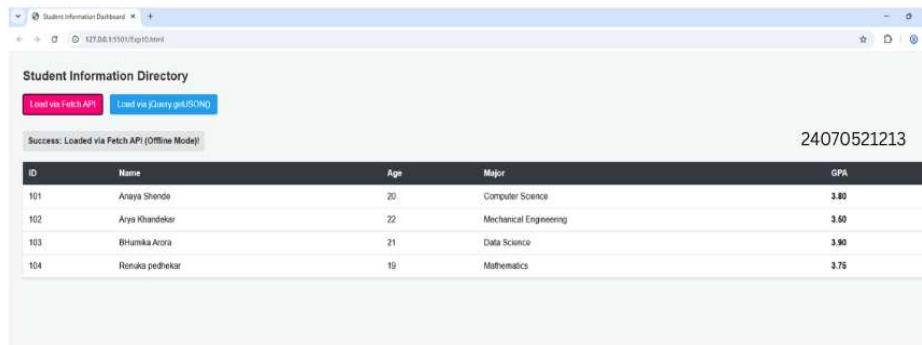
The **Fetch API** is a modern JavaScript API used to request data from files or servers. It returns a Promise that can be used to process the response asynchronously.

The `response.json()` method converts the JSON response into JavaScript data that can be processed by the program.

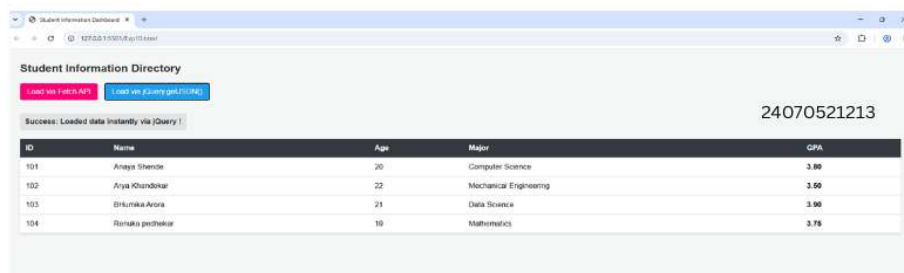
jQuery `getJSON()` is another method for loading JSON data. It simplifies the process of requesting and processing JSON files.

After receiving the JSON data, JavaScript can iterate through each student object and dynamically create HTML table rows. This avoids manually entering each row in the HTML and allows the table to update whenever the JSON data changes.

OUTPUT -



ID	Name	Age	Major	GPA
101	Anaya Shende	20	Computer Science	3.80
102	Arya Khandekar	22	Mechanical Engineering	3.50
103	Bhramika Arora	21	Data Science	3.90
104	Ronika pedhekar	19	Mathematics	3.75



ID	Name	Age	Major	GPA
101	Anaya Shende	20	Computer Science	3.80
102	Arya Khandekar	22	Mechanical Engineering	3.50
103	Bhramika Arora	21	Data Science	3.90
104	Ronika pedhekar	19	Mathematics	3.75

Conclusion - Thus, JSON data was successfully loaded using both the **Fetch API** and **jQuery getJSON()**. The retrieved student information was dynamically rendered in an HTML table, demonstrating asynchronous data fetching and dynamic manipulation of webpage content.